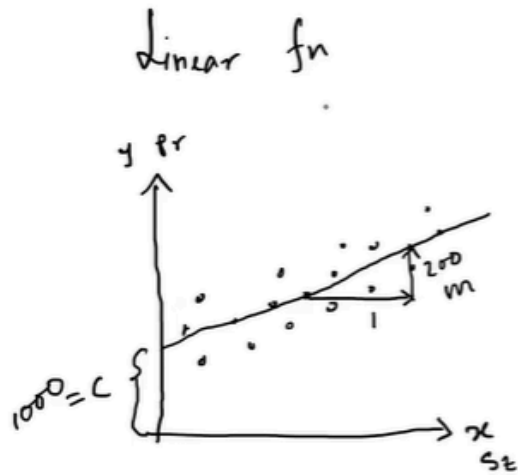


Video 1



Sizes, x Prices, y

$$y = a + b_1 x$$

$$= b_0 + b_1 x$$

$$= \beta_0 + \beta_1 x$$

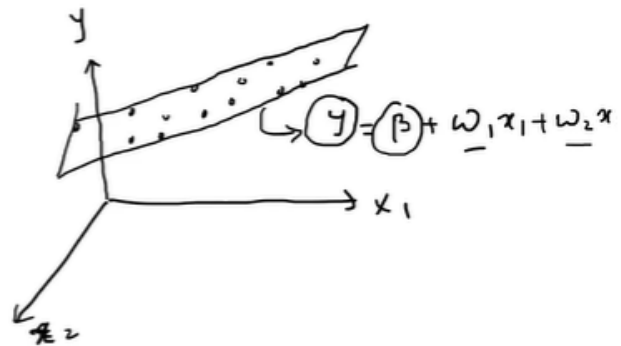
$$y = \textcircled{c} + \textcircled{m} x$$

$$= \underline{1000} + \underline{200} x$$

$$= \beta + \omega x$$

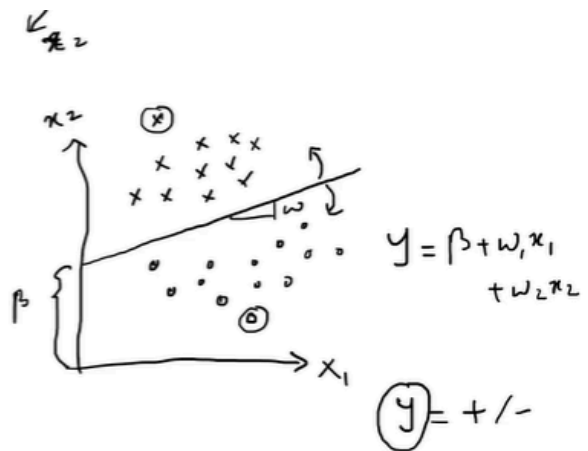
$$y = f(x)$$

x_1	x_2	y
size	rooms	pr
-	-	-
-	-	-



classification

x_1	x_2	y
-	-	1
-	-	0
-	-	0
-	-	1



Video 2

$$y_1 = \beta_1 + \omega_{11}x_1 + \omega_{12}x_2 + \omega_{13}x_3$$
$$y_2 = \beta_2 + \omega_{21}x_1 + \omega_{22}x_2 + \omega_{23}x_3$$

We can write this in **matrix form** as (we use **bold** font for representing vector and matrix variables):

$$\mathbf{y} = \mathbf{\beta} + \mathbf{\Omega}\mathbf{x}$$

Where:

- Input vector $\mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

- Weight matrix $\mathbf{\Omega}$:

$$\mathbf{\Omega} = \begin{bmatrix} \omega_{11} & \omega_{12} & \omega_{13} \\ \omega_{21} & \omega_{22} & \omega_{23} \end{bmatrix}$$

- Bias vector $\mathbf{\beta}$:

$$\mathbf{\beta} = \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix}$$

- Output vector $\mathbf{y}$:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

```
# Inputs as vector
x_vec = np.array([[x1], [x2], [x3]])

# Weight matrix
omega_mat = np.array([
    [omega11, omega12, omega13],
    [omega21, omega22, omega23]
])

# Bias vector
beta_vec = np.array([[beta1], [beta2]])

# Compute matrix-vector output
y_vec = beta_vec + np.matmul(omega_mat, x_vec)
```

Scalar (0D):

$$x = 5$$

Vector (1D):

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Matrix (2D):

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Tensor (3D Example):

Think of this as multiple matrices stacked together

$$\mathcal{T} = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} & \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \end{bmatrix}$$

$2 \times 2 \times 2$

```
import torch

# Input vector (3 features)
x_vec = torch.tensor([[4.0], [-1.0], [2.0]])

# Weight matrix (2 outputs, 3 inputs)
omega_mat = torch.tensor([
    [-1.0, 0.4, -0.3],
    [0.1, 0.1, 1.2]
])

# Bias vector (2 outputs)
beta_vec = torch.tensor([[0.5], [0.2]])

# Compute output using matrix-vector form
y_vec = beta_vec + torch.matmul(omega_mat, x_vec)
```

Attribute	Description	Example
.dtype	Data type of tensor elements (e.g., float32, int64)	torch.float32
.shape	Dimensions of the tensor (rows, columns, etc.)	(2, 3)

Element-wise Addition

Given two vectors:

$$\mathbf{a} = [1, 2, 3], \quad \mathbf{b} = [4, 5, 6]$$

Element-wise addition:

$$\mathbf{a} + \mathbf{b} = [1 + 4, 2 + 5, 3 + 6] = [5, 7, 9]$$

Element-wise Multiplication

Element-wise multiplication (also called Hadamard product):

$$\mathbf{a} \odot \mathbf{b} = [1 \times 4, 2 \times 5, 3 \times 6] = [4, 10, 18]$$

Dot Product (Inner Product)

The dot product of two vectors:

$$\text{dot}(\mathbf{a}, \mathbf{b}) = (1 \times 4) + (2 \times 5) + (3 \times 6) = 4 + 10 + 18 = 32$$

Matrix Multiplication

Given two matrices:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Matrix multiplication:

$$\mathbf{A} \times \mathbf{B} = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

```
import torch

# Element-wise addition
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
print("Element-wise addition:")
print(a + b)

# Element-wise multiplication
print("\nElement-wise multiplication:")
print(a * b)

# Dot product
print("\nDot product:")
print(torch.dot(a, b))

# Matrix multiplication
A = torch.tensor([[1, 2], [3, 4]])
B = torch.tensor([[5, 6], [7, 8]])
print("\nMatrix multiplication:")
print(torch.matmul(A, B))
```

- A matrix X of shape (2,3):

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

- A vector b of shape (3,):

$$b = [7, 8, 9]$$

When we do:

$$X + b$$

The vector b is **automatically expanded** (broadcasted) to match the shape of X .

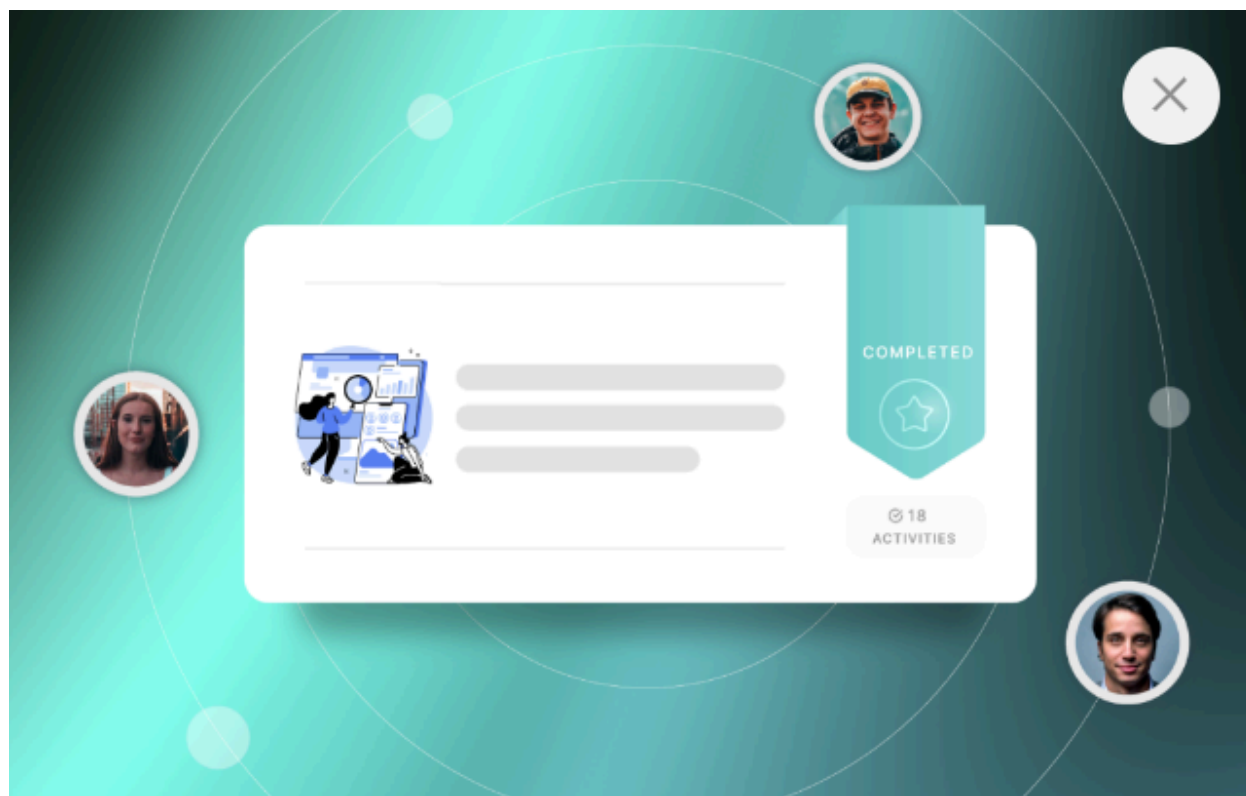
Result:

$$\begin{bmatrix} 1+7 & 2+8 & 3+9 \\ 4+7 & 5+8 & 6+9 \end{bmatrix} = \begin{bmatrix} 8 & 10 & 12 \\ 11 & 13 & 15 \end{bmatrix}$$

```
# Broadcasting example
X = torch.tensor([[1, 2, 3], [4, 5, 6]])
bias = torch.tensor([10, 20, 30])
```

```
# Broadcasting addition
print("Broadcasting result:")
print(X + bias)
```

```
Broadcasting result:
tensor([[11, 22, 33],
        [14, 25, 36]])
```



Congrats!

Project is complete!

Continue